

# STAT:5400 Section 2.2 — A First Look at Python

Boxiang Wang

2026-09-02

## 1 A First Look at Python

### 1.1 Python

- Python is a **general purpose** programming language.
- Python has simplified syntax and it is **easy** to learn and implement.
- Python has an active and mature **supporting community**.
- Python has many popular and powerful **libraries**, e.g., NumPy, Pandas, etc.
- Python was written by [Guido van Rossum](#) and the name was inspired by a comedy series “Monty Python”.



## Guido van Rossum - Personal Home Page



["Gawky and proud of it."](#)

### Who I Am

Read my ["King's Day Speech"](#) for some inspiration.

I am the author of the [Python](#) programming language. See also my [resume](#) and my [publications list](#), a [brief bio](#), assorted [writings](#), [presentations](#) and [interviews](#) (all about PyT Twitter).

I am retired, working on personal projects (and maybe a book). I have worked for Dropbox, Google, Elemental Security, Zope Corporation, BeOpen.com, CNRI, CWI, and

### How to Reach Me

You can send email for me to [guido \(at\) python.org](mailto:guido@python.org). I read everything sent there, but I receive too much email to respond to everything.

### My Name

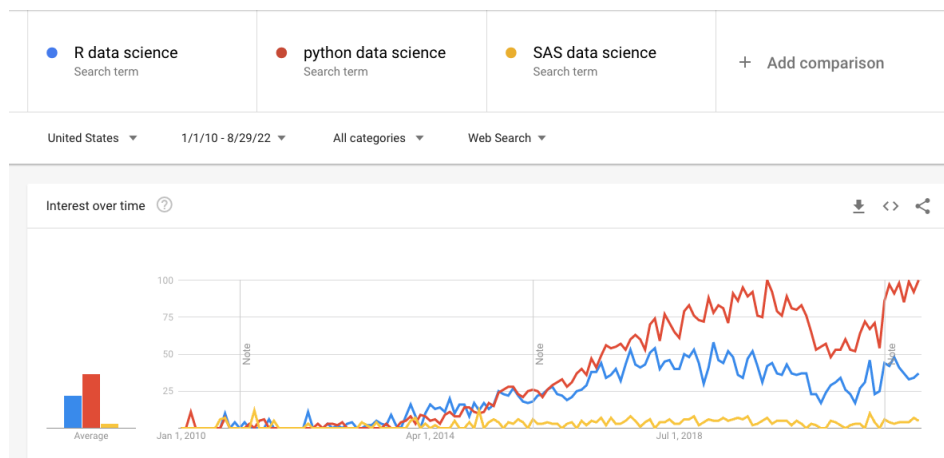
My name often poses difficulties for Americans.

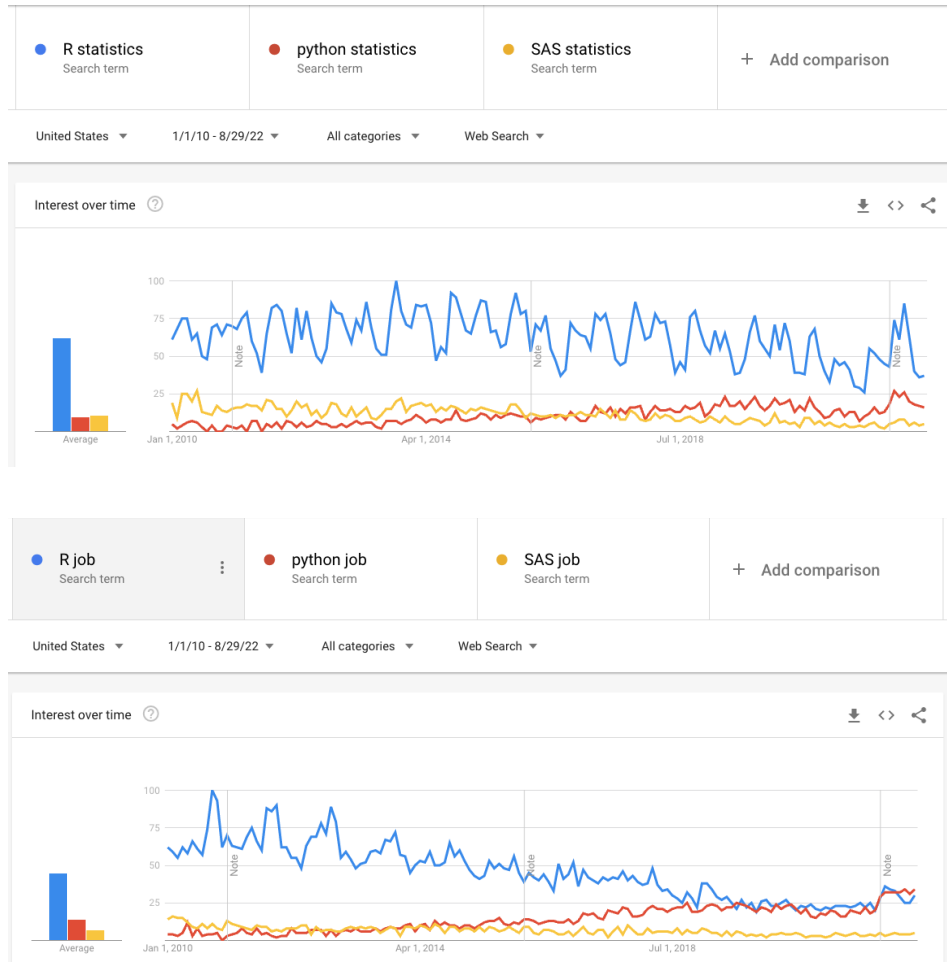
**Pronunciation:** in Dutch, the "G" in Guido is a hard G, pronounced roughly like the "ch" in Scottish "loch". (Listen to the [sound clip](#).) However, if you're American, you may assassinate some people have. :-)

**Spelling:** my last name is two words, and I'd like to keep it that way, the spelling on some of my credit cards notwithstanding. Dutch spelling rules dictate that when used in name is used alone to refer to me, it is capitalized, for example: "As usual, Van Rossum was right."

**Alphabetization:** in America, I show up in the alphabet under "V". But in Europe, I show up under "R". And some of my friends put me under "G" in their address book...

## 1.2 R and Python on Google Trends





### 1.3 Download Python

Two ways to install Python. Section 1.1 recommends **uv**, and that is still the recommendation here.

- **uv** — one small program that installs Python itself and manages the packages of each project separately. It is fast, and one project cannot break another, because each has its own copy of what it needs.
- **Anaconda** — a large bundle that ships Python together with hundreds of scientific packages and a graphical launcher. Nothing to choose: everything is already installed. It takes several gigabytes of disk and is slower to update.

Use **uv** if you are comfortable typing commands in a terminal. Use **Anaconda** if you would rather install everything at once and start from a window. Either works for this course.

Recommended integrated development environment (IDE): (advanced editor that can be used to build and execute code)

- Jupyter Notebook.
- Spyder.
- PyCharm.
- VS Code.

Nothing to install for this page: the blocks below run Python in your browser, and the course Codespace already has Python, Jupyter, and VS Code.

## 1.4 Hello 5400!

```
print('Hello', '5400!')  
print('Hello', '5400', end='!\n')  
print('Hello' + ' ' + '5400')
```

```
print(paste("Hello", "5400!"))
```

## 1.5 Basic operations in R

```
1 + 1  
exp(-2)  
sqrt(2 * pi)
```

```
2 / 0  
3 / 2 * 3
```

## 1.6 Basic operations in Python

In Python, `exp` and `sqrt` are not there until you import them.

```
1 + 1
```

The next block is meant to fail.

```
exp(-2)
```

Let's import some math functions from the `math` module (library).

```
import math
from math import *
```

Or directly import the functions that are needed.

```
from math import exp, sqrt, pi
print(exp(-2))
print(sqrt(2 * pi))
```

Dividing by zero is an error in Python, not `Inf`. This block is meant to fail.

```
2 / 0
```

## 1.7 Integers and floats

In Python 3, `/` is float division and `//` is integer division.

💡 Predict — one of these three lines prints something different from the other two. Which one?

```
print(3 / 2 * 3)
print(3 // 2 * 3)
print(3.0 / 2 * 3)
```

Python 2 used `/` for integer division, so the same line printed `3` in the old notes.

`R` converts integers to float values as needed.

```
3 / 2 * 3
3L / 2L * 3L
```

## 2 Data Analysis

### 2.1 Reading the data

```
Cars <- read.delim("data/Cars.dat")
# <- is assignment operator
str(Cars)
```

## 2.2 The library pandas

Quote from its official website: <https://pandas.pydata.org/>

*pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.*

pandas provide abundant functions to import, index, manipulate, merge, and plot data frame.

**Run this block first** — the Python blocks below use `Cars`.

```
import pandas as pd
Cars = pd.read_csv('data/Cars.dat', sep='\t')
Cars.dtypes
```

## 2.3 The first rows

```
# show the first two rows
head(Cars, 2)
```

```
Cars.head(2)
```

## 2.4 Summaries

```
summary(Cars[, c("Country", "MPG")])
```

```
Cars[['Country', 'MPG']].describe(include='all')
```

## 2.5 All pairs

```
plot(Cars)
```

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.pairplot(Cars, hue='Country')
plt.show()
```

## 2.6 Data frames

The object `Cars` is a data frame.

```
class(Cars)
```

```
type(Cars)
```

Data frame:

- Special kind of object.
- Like a table of data.
  - Row for each observation.
  - Column for each variable.
- Variables (columns) can be of different data types:
  - Numeric, character, factors.

## 2.7 Vectors

Four basic types of variables: integer, double, logical, character.

```
class(1L)
class(1)
class(TRUE)
class("TRUE")
```

```
print(!TRUE)
class(complex(1, 1, -1))
```

```
print(type(1))
print(type(1.0))
print(type(True))
print(type('True'))
```

```
print(not True)
print(type(1 - 1j))
```

## 2.8 Vectors and lists

```
x = c(5, 4, 0, 0)
x
```

```
y = c("Hello", "5400")
y
c(y, "!")
```

```
x = [5, 4, 0, 0]
print(x)
```

```
y = ['Hello', '5400']
print(y)
print([y, '!'])
```

```
y.append('!')
print(y)
```

👉 By hand — this difference catches R users more than any other. A chatbot was asked whether `b = a` copies a list. Its answer is in the comments. Run the R block, then the Python block, and check the claim against both. Done when you can say what each one prints, and why they differ. No AI for this block.

```
# The same two lines in R, for comparison.
a <- c(1, 2, 3)
b <- a
b[4] <- 4
a
```

```
# Chatbot: "b = a makes b a copy of the list, so the two are independent.
# Appending to b leaves a unchanged, and a still prints [1, 2, 3]."
```

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

### **i** Solution

**R prints 1 2 3. Python prints [1, 2, 3, 4].**

In R, changing **b** makes a copy first, so **a** is untouched. In Python, **b = a** does not copy the list; it gives the one list a second name, so **a is b** is **True** and appending through either name changes it. Ask for the copy: **b = a.copy()**.

So the chatbot's answer describes R, not Python. That is the trap: it is the right answer to the wrong language.

A list in R and python can contain variables of different types.

```
z = list("word" = c("Hello", "5400"), "sign" = "!")
z$"word"
z$"sign"
```

```
y = ['Hello', '5400']
z = {'word': y, 'sign': '!'}
print(z)
```

```
print(z['word'])
print(z['sign'])
```

## 2.9 Matrices and arrays

In R, **byrow=FALSE** is by default when assembling a matrix.

```
a <- matrix(1:6, ncol = 3, nrow = 2, byrow = FALSE)
a
```

```
b <- c(1, 2, 3)
a %*% b
```

In Python, we can use **order='F'** to convert a vector to a matrix by column; otherwise, the conversion is by row.

```
import numpy as np
a = np.reshape(list(range(1, 7)), (2, 3), order='F')
a
```

```
b = [1, 2, 3]
np.dot(a, b)
```

The `range` function in Python has arguments: `start`, `stop`, `step`.

## 2.10 The library Numpy

Quote from its official website: <https://numpy.org/> *the fundamental package for scientific computing*.

- It is mainly devised for processing and operating arrays with general purpose.
- It provides functions to add, multiply, slice, reshape, broadcast, and index, arrays.

```
1:4 * 3
```

💡 Predict — the same `* 3`, on a list and on a NumPy array. Do they do the same thing?

```
import numpy as np
print(list(range(1, 5)) * 3)
print(np.array(range(1, 5)) * 3)
```

A Python list repeats when you multiply it; a NumPy array multiplies element by element. In Python 2, `range(1, 5)` was already a list, which is what the old notes show.

## 3 Subsetting

### 3.1 Subsetting a vector

```
x = 1:5 * 3
x[1:2]
```

```
x[seq(1, 5, 2)]
x[c(1, 3)]
```

```
x = np.array(range(1, 6)) * 3
print(x)                # range [start, stop) -- a half-open interval
```

💡 Predict — how many elements does each line return? Compare with the R block above.

```
print(x[0:1])          # note Python begins with 0
print(x[0:4:2])
print(np.array(x)[[0, 2]])
```

👉 By hand — a chatbot was asked what happens when a logical index is shorter than the vector. Its answer is in the comments. Run the R block, then the Python block. One language does something surprising, the other refuses to guess. Done when you can say what R returns, and what NumPy says instead. No AI for this block.

```
# Chatbot: "A logical index shorter than the vector is padded with FALSE, so
# only the first element is kept. R and Python both behave this way."
v <- 1:6 * 5
v[c(TRUE, FALSE)]
```

```
import numpy as np
v = np.array(range(1, 7)) * 5
try:
    print(v[np.array([True, False])])
except IndexError as e:
    print("IndexError:", e)
```

### **i** Solution

**R returns 5 15 25. NumPy returns an error.** The chatbot is wrong twice. R does not pad with FALSE. It recycles the index to the length of `v`, giving TRUE FALSE TRUE FALSE TRUE FALSE, so three elements come back, not one. NumPy refuses: `boolean index did not match indexed array along axis 0; size of axis is 6 but size of corresponding boolean axis is 2. The mask must be the same length as the array.` This is worth remembering in both directions. R guesses what you meant and stays silent, which is convenient until the guess is wrong. NumPy makes you say it exactly. To take every other element in Python, slice instead: `v[::2]`.

👉 By hand — a chatbot was asked whether it is safe to modify a slice of a NumPy array. Its answer is in the comments, and one claim in it is wrong. Run the block. Done when you can say which of the three lines changed `a`, and why that one and not the others. No AI for this block.

```
# Chatbot: "Slicing a NumPy array returns a copy, just as R does. Writing to
# the slice never touches the original, so a prints [1 2 3 4 5] all three
# times below."
```

```

import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = a[0:2]
b[0] = 99
print("after a[0:2]  :", a)

a = np.array([1, 2, 3, 4, 5])
c = a[[0, 1]]
c[0] = 99
print("after a[[0,1]] :", a)

a = np.array([1, 2, 3, 4, 5])
m = a[a > 3]
m[0] = 99
print("after a[a > 3] :", a)

```

```

# The same idea in R, for comparison.
a <- c(1, 2, 3, 4, 5)
b <- a[1:2]
b[1] <- 99
a

```

### **i** Solution

**Only the first line changed `a`.** It prints `[99 2 3 4 5]`. The other two leave `a` alone, and so does R.

`a[0:2]` returns a view: a second way of reading the same memory. Writing through it writes into `a`.

`a[[0, 1]]` and `a[a > 3]` return copies. Not by choice: a view can only describe positions at a regular step — a start, then every `k`-th element. A list of positions, or whatever a mask happens to select, is not regular, so NumPy has to gather those values into new memory.

Slicing is a view because copying is expensive. Taking `big[0:5000000]` as a copy has to move 40 MB; as a view it moves nothing. R never does this: `a[1:2]` always allocates. So the bug cannot happen in R, and an R user has no habit of watching for it.

Two ways to stay safe: write `a[0:2].copy()` when you mean to modify, and check with `np.shares_memory(a, b)` when you are unsure. Note that a plain Python list is not an array — `lst[0:2]` always copies — so testing the idea on a list teaches the wrong rule.

## 3.2 Subsetting a matrix

```
a = matrix(1:8, 2, 4)
a[1, ]
a[, 1]
```

```
colMeans(a)
rowMeans(a)
```

```
a = np.array([[1, 3, 5, 7], [2, 4, 6, 8]])
print(a[0, ])
print(a[:, 0])
```

💡 Predict — one of these matches `colMeans` above. Axis 0 or axis 1?

```
print(a.mean(axis=0))
print(a.mean(axis=1))
```

## 3.3 Strings in Python

```
s = 'Hello 5400'
print(len(s))
print(s[0])
```

```
print(s[0:6:2])
print(s[-1])
print(s[6:-1])
```

```
print(s.lower()) # s.upper()
```

```
a, b = s.split(' ')
print(a)
print(b)
print(a + b)
```

## 4 A Simple Statistical Analysis

### 4.1 Selecting columns

```
head(Cars[, c("Country", "MPG", "Weight")])
```

```
Cars[['Country', 'MPG']].head(5)
```

### 4.2 Selecting rows

```
US_Cars = Cars[Cars$Country == "U.S.", ]  
head(US_Cars[, c("Country", "MPG")], 5)
```

```
US_Cars = Cars.loc[Cars['Country'] == 'U.S.']  
US_Cars[['Country', 'MPG']].head(5)
```

```
JPN_Cars = Cars.loc[Cars['Country'] == 'Japan']  
JPN_Cars[['Country', 'MPG']].head(5)
```

### 4.3 Counts and summaries

```
table(Cars$Country)  
summary(Cars[Cars$Country == "U.S.", "MPG"])  
summary(Cars[Cars$Country == "Japan", "MPG"])
```

```
from collections import Counter  
Counter(Cars['Country'])
```

```
pd.concat([US_Cars['MPG'],  
          JPN_Cars['MPG']], axis=1).describe()
```

### 4.4 Histograms

```
hist(Cars[Cars$Country == "U.S.", "MPG"])
```

```
import matplotlib.pyplot as plt
plt.hist(US_Cars['MPG'], bins=range(15, 40, 5))
plt.show()
```

## 4.5 The package Matplotlib

Quotes from its official website <https://matplotlib.org/>.

- *Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.*
- *Matplotlib makes easy things easy and hard things possible.*

Provides functions to produce scatter plots, bar charts, histogram, box plots, pie charts, etc.

```
hist(Cars[Cars$Country == "Japan", "MPG"])
```

```
import matplotlib.pyplot as plt
plt.hist(JPN_Cars['MPG'], bins=range(15, 40, 5))
plt.show()
```

## 4.6 One-sample t-test

Test if the mean MPG of the U.S. cars is 20.

```
t.test(US_Cars[, "MPG"], mu = 20)
```

```
from scipy.stats import ttest_1samp
t1 = ttest_1samp(US_Cars['MPG'], 20)
print(t1)
```

```
print(t1.pvalue)
```

## 4.7 Two-sample t-test

```
JPN_Cars = Cars[Cars$Country == "Japan", ]  
t.test(US_Cars[, "MPG"], JPN_Cars[, "MPG"])
```

```
from scipy.stats import ttest_ind  
t2 = ttest_ind(US_Cars['MPG'], JPN_Cars['MPG'],  
               equal_var=False)  
print(t2)
```

```
print(t2.pvalue)
```